Diseño de Algoritmos – Práctica 0

Introduction to GNU gprof

Francisco Palomo-Lozano

Version 2.2

Contents

1 Introduction **3**

2 Installing the software **3**

3 Basic concepts **4**

4 Using the GNU profiler **5**

5 A step-by-step example **8**
5.1 Analyzing the flat profile 9
5.2 Analyzing the call graph 11
5.2.1 Main line 13
5.2.2 Parent function line 13
5.2.3 Child function line 14
5.2.4 Mutually recursive functions 14

6 Visualizing the call graph **15**
6.1 Installing the tools 15
6.2 Using the tools 15
6.3 Interpreting the output 16
6.4 Example 17

1 Introduction

GNU gprof is an open source *profiling tool* that provides information about program execution. For example, thanks to this tool, we can know how many times each line of code in a program is executed, or how long it takes to execute a function. This way, we can detect bottlenecks, untested code, or just focus our optimization efforts on those parts of the program that are more time consuming.

Nowadays, it is not uncommon to find large software systems in industry consisting of many small modules. These systems are usually tied to major responsibilities, such as ship and aircraft control or production line management, where performance is important. Therefore, it is possible that we face to a large system structured in several layers where inefficiencies may arise in many different locations.

Besides, it is quite common that a high percentage of the execution time is due to just a few functions, which represent a small percentage of the whole source code. At this point, it may be desirable to analyze its performance to detect where most of the execution time is being consumed and try to improve the performance of the relevant portions of code, if possible.

Thanks to the information provided by `gprof`, we can also evaluate alternative implementations of a program. Maybe we want to assess the impact of a new algorithm or decide on whether introducing a different data structure will pay off. GNU gprof provides two different kinds of reports that can help us in this tasks: the *flat profile* and the *call graph*. We will describe them later in this document.

2 Installing the software

GNU gprof is part of GNU Binutils[1], which is a collection of *binary tools* that it is included by default in most GNU/Linux distributions. However, if it is not installed in our system, or we want to update it, we can install GCC (the GNU Compiler Collection), which also depends on other useful Binutils tools.

The following privileged session shows how to install or update GCC to the latest available version in the repositories configured in our distribution, including the GNU C and GNU C++ compilers `gcc` and `g++`, respectively, `gprof`, `make`, and other utilities:

```
$ sudo -i
password for user: ********
# apt update && apt upgrade
  ...
# apt install build-essential
  ...
# exit
```

[1] https://www.gnu.org/software/binutils

The above instructions will certainly work in any Debian-based GNU/Linux distribution, like Ubuntu. Please, check your distribution for instructions on how to install packages and which packages are available, as packaging may be different.

3 Basic concepts

Before diving into `gprof` details, it is important to review some basic concepts. We have mentioned that `gprof` is a *profiling tool*, or *profiler*. The purpose of a profiler is to enable *code profiling*: a technique that allows conducting performance analysis by instrumenting either the program source code or the corresponding executable binary. In particular, `gprof` is a *binary profiler*, which instruments the executable binary, so there is no need to modify the source code. Although, we will focus on `gprof` and C/C++ code performance analysis, profiling has a broad range of applications, such as:

- Function tracing.
- Thread profiling.
- Coverage analysis.
- Memory profiling (heap profiling and garbage collection).
- Analysis of external resource invocation (web services, databases, etc.).

Profilers collect information during the actual execution of the program. Because of this dynamic nature, profilers cannot analyze what is not executed. If, in a run of the program, the control flow does not go through a certain function, no information will be provided about it. *The data the program is run on is as important as the program itself*. Moreover, as the executable code is instrumented, it is important to take into account external factors that may impact performance measures (the operating system and other running processes, the system load, etc.).

Profiling has three fundamental steps, to be executed sequentially:

1. *Compiling and linking*. The program has to be compiled and linked with profiling enabled. As mentioned before, `gprof` is a binary profiler and we do not need to instrument the source code for profiling. However, we must instruct the compiler to instrument the object code, and the linker to use the profiling libraries. Both goals can be accomplished by supplying the option `-pg` during compilation and linking. In the following example, all modules are compiled with this option and then linked together to produce the executable binary program:

   ```
   $ gcc -pg -c main.c module-*.c          # Compiling.
   $ gcc -pg -o program main.o module-*.o  # Linking.
   ```

   Of course, a single compiler invocation that both compiles and links suffices:

   ```
   $ gcc -pg -o program main.c module-*.c  # Compiling & linking.
   ```

2. *Program execution.* The program can be run as usual, though it may take longer because its executable binary is now instrumented and collecting the data required for profiling during execution requires some time. Once the program is executed, if it finishes its execution normally, a file with the relevant data is produced. In our case, we get a file named `gmon.out` that we can rename or move to another directory according to our needs:

   ```
   $ ./program
   $ mv gmon.out program-profile.out
   ```

   It can be interesting to execute several runs of the same program, saving each profile data file with a different name for later analysis. On the one hand, several runs with the same data can help to cater for variability stemming from system load and other external factors, or by non-deterministic behavior. By averaging different results, *more precise profiles* can be obtained. On the other hand, several runs with different data can help to cover the execution of different functions or regions of code, so that *more complete profiles* can be obtained.

3. *Profiler invocation.* Once the profiling data has been generated, the profiler can be invoked to interpret the results. In the case of `gprof`, it receives the executable binary and the profile data files and writes a flat profile and call graph through the standard output. Therefore, it is advisable to redirect the standard output to a file for further analysis:

   ```
   $ gprof program program-profile.out > program-profile.txt
   ```

Warning! When compiling and linking separately, we must make sure that all modules have been processed with `-pg`. If a module is not compiled with `-pg`, we will only obtain information about the total execution time of its functions, without any additional information. In particular, it will be impossible to generate the *call graph* for them. If this happens, the following error message will be produced upon running `gprof`:

```
gprof: gmon.out file is missing call-graph data
```

4 Using the GNU profiler

In order to use `gprof` effectively some basic recommendations should be followed:

1. Optimization should be the last step in program development. It has no sense to devote effort to improve the performance of an incorrect program.

2. Always compile and link with the `-pg` option. When using `make`, this implies specifying this option through the `CFLAGS` (for C), `CXXFLAGS` (for C++), and `LDFLAGS` variables during its invocation, or inside the *makefile* for a permanent use.

3. Run your program with the appropriate command-line arguments and inputs. Remember that you profile a program execution, that is, a particular run that

depends on the data that you supply to your program.

4. Name the resulting files properly and unambiguously, so that no misunderstanding about their contents may arise later.
5. Run the profiler and pay close attention to its output, trying to understand what it means. When in doubt, read the manual.[2]

The invocation has the following syntax, where optional arguments are enclosed in square brackets:

`gprof` [*options*] [*executable* [*profile* ...]] [> *output*]

If the executable filename is omitted, `a.out` is used by default. If no profile filenames are provided, `gmon.out` is used. As you can see, you can supply more than one profile data file, obtaining a single output corresponding to all the data being tallied together.

GNU gprof includes many configuration options to control the output. Many of them can include special arguments called *symspecs* or *specification symbols*. These symbols are used to specify files, functions, or line numbers for the different options. A *symspec* consists of one of the following elements:

1. A filename containing a '.' character.
2. A function name.
3. A line number.
4. A filename, followed by a ':' character and a function name or a line number.

For example, `program.c:main` is a *symspec* which matches the `main` function symbol in file `program.c`.

If a *symspec* is specified alongside an option, the action described by the option is only applied to the symbols matched by the *symspec*. Options can be included multiple times in the command line with different *symspecs*.

The following options are used often:

`-b` | `-brief`
Do not show the meaning of each field in the tables.

`-h` | `-help`
Show command-line usage.

`-s` | `-sum`
Merge the profile data into file `gmon.sum`.

`-v` | `-version`
Show version.

[2] https://sourceware.org/binutils/docs/gprof

-demangle[=*style*]
Demangle C++ symbol names according to the optional compiler-dependent style.

-no-demangle
Do not demangle C++ symbol names.

-p[*symspec*] | **-flat-profile**[=*symspec*]
Show the flat profile.

-P[*symspec*] | **-no-flat-profile**[=*symspec*]
Do not show the flat profile.

-q[*symspec*] | **-graph**[=*symspec*]
Show the call graph.

-Q[*symspec*] | **-no-graph**[=*symspec*]
Do not show the call graph.

By default, when no options are specified, the flat profile and the call graph are shown. The profiling data that `gprof` yields includes the running time and number of calls corresponding to each function that is executed in a run of the program.

The running time is based on a *sampling process*, which probes the *program counter* at regular periods of time to estimate its value. Therefore, times measured are subject to statistical error too and may vary from run to run. In contrast, the number of calls is accurate and does not change, as it is measured by counting instead of sampling. However, this is only guaranteed if the program is deterministic and single-threaded. For example, when a program calls a function depending on its environment (command-line arguments, input data, random numbers, or other external state), runs more than one execution thread, or uses multi-threaded libraries, `gprof` can certainly produce different results each run.

One way to get more accuracy is to make your program run longer. For example, you could modify some internal parameters or increase the size of its inputs. However, this is not always feasible. Another way is to combine the data from several runs:

1. Run the program once.
2. Rename `gmon.out` to `gmon.sum`.
3. Repeat as many times as desired:
   3.1. Run the program again.
   3.2. Merge the new `gmon.out` into `gmon.sum`.
4. Analyze the resulting profile data in `gmon.sum`.

It is important to make sure beforehand that the results for each run are comparable. This can be obvious for programs exhibiting a deterministic behavior if we keep exactly the same conditions for each run, but can be difficult, or even impossible, otherwise.

5 A step-by-step example

In this section, we are going to work out step by step an example of profiling with `gprof`, focusing on how to interpret the resulting information. The code to profile is a simple C program whose purpose is to repeat, through several nested loops, a SAXPY floating-point benchmark interspersed with a busy-wait delay. The source code is stored in a single file, `main.c`, with the following content:

```c
#include <stdio.h>

// Arrays for the SAXPY benchmark.

#ifndef N
#define N 4096
#endif

float x[N], y[N];

// SAXPY loop.

void saxpy(float a, float *restrict x, float *restrict y, int n)
{
  for (int k = 0; k < n; ++k)
    y[k] = a * x[k] + y[k];
}

// Busy-wait delay.

static inline void delay(void)
{
  for (int k = 0; k < 2000; ++k)
    ;
}

// Benchmark.

void benchmark(int n)
{
  for (int k = 0; k < n; ++k) {
    saxpy(0.5, x, y, N);
    delay();
  }
}

```

```
37 // Loop to repeat the benchmark.
38
39 void loop(int n)
40 {
41   for (int k = 0; k < n; ++k)
42     benchmark(n);
43 }
44
45 // Benchmark initialization and start up.
46
47 const int n = 1000;
48
49 int main(void)
50 {
51   for (int k = 0; k < N; ++k) {
52     x[k] = 0.001;
53     y[k] = 0.01;
54   }
55   loop(n);
56   return 0;
57 }
```

First, we compile and execute the program to generate the file `gmon.out`:

```
$ gcc -pg -o main main.c
$ ./main
  ...
```

Then, we run `gprof`, hiding the explanatory information with the `-b` option once we get familiarized with the output format:

```
$ gprof -b main > profile.txt
```

In order to separate the flat profile from the call graph into different files, we can instead run `gprof` with the appropriate options:

```
$ gprof -bp main > flat-profile.txt
$ gprof -bq main > call-graph.txt
```

In the following sections we will analyze the output generated by this execution. Please, note that the real data that you obtain may vary depending on all the factors described above.

5.1 Analyzing the flat profile

The output corresponding to the flat profile follows:

```
Flat profile:

Each sample counts as 0.01 seconds.
  %   cumulative   self              self     total
 time   seconds   seconds    calls   s/call   s/call  name
 76.97      9.10     9.10  1000000     0.00     0.00  saxpy
 23.36     11.86     2.76  1000000     0.00     0.00  delay
  0.08     11.87     0.01     1000     0.00     0.01  benchmark
  0.00     11.87     0.00        1     0.00    11.87  loop
```

The flat profile is layout in a table. Each entry in the table contains information about one function. Entries appear sorted by decreasing time in *self seconds*, then by decreasing number of *calls*, then alphabetically by *name*. The sampling period is shown. Thus, a sampling period of 0.01 seconds means that 100 samples per second were collected during the execution of the instrumented binary executable. The fields in the table have the following meaning:

% time
Percentage of the running time spent on this function.

cumulative seconds
Time spent on this function plus time spent on the functions above.

self seconds
Time spent on this function.

calls
Number of calls that were made to this function.

self s/calls
Average time per call spent on this function.

total s/calls
Average time per call spent on this function, and the functions it calls.

name
Name of the function.

Once the meaning of the fields are known, it is not difficult to interpret the results. The program execution took 11.87 s with almost all the running time spent on `saxpy` ($\approx 77\%$) and `delay` ($\approx 23\%$). Each of them has been called 1000000 times, as function `benchmark` has been called 1000 times. In contrast, function `loop` has been called once, though this call takes practically all the running time.

As mentioned before, running times are subject to error. Times shown are regarded accurate if they are considerably longer than the sampling period. Thus, when a time is shown as zero, it is not really zero, but negligible.

The magnitude of the error can be predicted. For n samples, the expected absolute error is $\epsilon = \sqrt{n} \cdot T$, where T is the sampling period. For a measure of time t this yields:

$$n = \frac{t}{T} \quad \text{and} \quad \epsilon = \sqrt{\frac{t}{T}} \cdot T = \sqrt{t \cdot T} \ .$$

For example, function `saxpy` takes $t = 9.10\,\text{s}$ and the sampling period is $T = 0.01\,\text{s}$, as indicated at the beginning of the flat profile. Therefore, $\epsilon = \sqrt{9.10 \cdot 0.01}\,\text{s} \approx 0.30\,\text{s}$, that is, a relative error of $(0.30 \cdot 100)/9.10 \approx 3.31\,\%$.

In order to reduce the errors and increase the accuracy of our measures, we can repeat the execution a number of times to gather more data, as explained in Section 4:

```
$ gcc -pg -o main main.c
$ ./main
  ...
$ mv gmon.out gmon.sum
$ for i in {1..9}
> do
>     ./main
>     gprof -s main gmon.out gmon.sum
> done
  ...
$ gprof -b main gmon.sum > profile.txt
```

In the above session, an arbitrary number of runs is used: 10 runs in total. In practice, the number of repetitions should be adjusted to the accuracy desired and the resources available. Nevertheless, the main goal of profiling is identifying those functions consuming most of the execution time to focus our efforts in improving them. When the execution times of these functions are big enough, there is no need to get more precise estimates or increasing the accuracy of measures for other functions whose execution time is marginal in comparison.

5.2 Analyzing the call graph

The output corresponding to the call graph follows:

```
                     Call graph


granularity: each sample hit covers 2 byte(s) for 0.08% of 11.87 seconds

index % time    self  children    called     name
                0.01   11.86    1000/1000        loop [2]
[1]    100.0    0.01   11.86    1000         benchmark [1]
                9.10    0.00 1000000/1000000     saxpy [4]
```

```
                2.76    0.00 1000000/1000000     delay [5]
-----------------------------------------------
                0.00   11.87       1/1           main [3]
[2]    100.0    0.00   11.87       1         loop [2]
                0.01   11.86    1000/1000        benchmark [1]
-----------------------------------------------
                                                 <spontaneous>
[3]    100.0    0.00   11.87                 main [3]
                0.00   11.87       1/1           loop [2]
-----------------------------------------------
                9.10    0.00 1000000/1000000     benchmark [1]
[4]     76.6    9.10    0.00 1000000         saxpy [4]
-----------------------------------------------
                2.76    0.00 1000000/1000000     benchmark [1]
[5]     23.3    2.76    0.00 1000000         delay [5]
-----------------------------------------------

Index by function name

   [1] benchmark               [2] loop
   [5] delay                   [4] saxpy
```

This is a tabular representation of a graph that shows the time spent on each function, the calls received from other functions (the *parents*) and the calls made to other functions (the *children*). This data is quite effective to detect functions which do not take long to execute themselves, but do invoke functions with long execution times.

There is an entry for each function. Entries are delimited by lines and consist of:

- The *main line*, starting with a number in square brackets, which is the *index* of this function.
- The *parent function lines*, which appear above the main line, one line per each parent function, if any.
- The *child function lines*, which appear below the main line, one line per each child function, if any.

Following the table, a reference with the index corresponding to each function is included. The entries are sorted in decreasing order by the joint time spent on the main function and its children.

If we focus on the first entry of our example, we can see that the main line is devoted to function `benchmark` (index 1), that is called from `loop` (index 2) and calls `saxpy` and `delay` (indices 4 and 5, respectively).

Next, we will describe the meaning of each field in the main line, the parent function

lines, and the child function lines, as well as how cliques of mutually-recursive functions are dealt with. All times are given in seconds.

5.2.1 Main line

The meaning of the fields for this line is as follows:

index
Number identifying the function in the call graph.

% time
Percentage of time spent on the function, including child functions.

self
Time spent on the function itself.

children
Time spent on child functions.

called
Number of calls to the function. In the presence of recursive calls, two numbers are displayed (separated by '+'): the number of external calls plus the number of recursive calls.

name
Function name followed by its index.

5.2.2 Parent function line

The meaning of the fields are different, as they are relative to a parent function that calls the main function described in the same entry and, in general, child functions described in other entries.

self
Time spent on the main function by this parent.

children
Time spent on child functions by this parent.

called
Two numbers are displayed (separated by '/'): the number of calls from the parent to the main function over the total number of calls to the main function excluding recursive calls.

name
Function name followed by its index.

If it is not possible to identify the parent function, all the fields will be empty, save for the name, which will display `<spontaneous>`. In our example, this happens with the

function *main* that is the entry point to our program.

5.2.3 Child function line

The meaning of the fields are different, as they are relative to a child function that is called by the main function described in the same entry and, in general, by parent functions described in other entries.

self
Time spent on this child function by the main function.

children
Time spent on the children of this child function by the main function.

called
Two numbers are displayed (separated by '/'): the number of calls from the main function to this child over the total number of calls to this child excluding recursive calls.

name
Function name followed by its index.

5.2.4 Mutually recursive functions

Sometimes, functions call themselves recursively, either directly or indirectly, inducing a *cycle in the call graph*. If a function calls itself, it is a *direct recursive call* with no other functions involved in the recursion. However, if a function calls another function that eventually calls the former, it is an *indirect recursive call*.

There may be several functions involved in a chain of indirect recursive calls. Consequently, there is a set of *mutually recursive functions* involved in a cycle. A function can be then both a parent and a child of itself. How can we assign time measures to functions in this context?

A solution is to deal with cycles as a whole. Recursive calls are then interpreted as calls internal to the cycle. The call graph can show an entry for each cycle. This entry can show instead who called any function in the cycle (as parents) and the functions in the cycle (as children). Functions in the cycle can show how many times they are called by functions outside the cycle (non-recursive calls) plus how many times they are called by functions inside the cycle (recursive calls).

This is the way `gprof` solves this problem. Cycles are numbered with consecutive integers and a special *cycle entry* is added for each cycle with name `<cycle n as a whole>`, `n` being the cycle number. When a function belongs to a cycle, each time the function name appears in the call graph it is followed by `<cycle n>`.

In a cycle entry, the main line describes the total time spent directly on the functions in the cycle. The parent lines represent the functions outside the cycle that called any

function in the cycle. The child lines represent the functions inside the cycle and their children.

In the entry corresponding to a function in a cycle, some functions in the same cycle may appear both as parents and children at the same time. In this case, their **self** and **children** fields are empty because of the difficulty of assigning time measures to such functions when recursion is going on. However, their **called** fields show how many times such functions called (as a parent) another function inside the cycle and were called (as a child) by another function inside the cycle.

6 Visualizing the call graph

One of the advantages of using `gprof` is that we can transform its output to the format that best suits our needs, thanks to several tools created with this purpose. One of this tools is `gprof2dot`, which allows us to visualize the dynamic call graph with profile data in a helpful and colorful way. This makes profiles easier to interpret and faster to analyze. The graph is translated into the DOT language, which is easily converted into many different graphic formats or rendered in a window.

6.1 Installing the tools

In addition to `gprof2dot`, we will install two utilities to process DOT files: the GraphViz visualization software[3] and the `xdot` interactive viewer[4] for the DOT language. Both, `gprof2dot` and `xdot`, requires Python, with `xdot` requiring Python 3. Installation instructions follow:

```
$ sudo -i
password for user: ********
# apt install graphviz python3-pip python3-numpy xdot
  ...
# pip install gprof2dot
# exit
```

The `pip` command above installs `gprof2dot` from the Python Package Index (PyPI). A local, unprivileged, installation of these packages is possible too with `pip`.

6.2 Using the tools

We can feed `gprof2dot` with `gprof` output, for example, through a pipe, or use its output previously saved in a file. Regardless of the method used, the result will be a graphic profile in DOT format. Again, this can be saved in a file or sent to the standard output, where it could be sent instead to other tools, like `dot` from the GraphViz collection, or saved for later use. The following session illustrates this:

[3] https://graphviz.org
[4] https://github.com/jrfonseca/xdot.py

```
$ ./main && mv gmon.out profile.out
  ...
$ gprof main profile.out > profile.txt
$ gprof2dot profile.txt > profile.dot
$ dot -Tpdf -o profile.pdf profile.dot
```

When no intermediate files are desired, a more succinct way is:

```
$ ./main
  ...
$ gprof main | gprof2dot | dot -Tpdf -o profile.pdf
```

In a system running the X Window System, the intermediate `.dot` file can be displayed by specifying the X11 protocol as a format for `-T`:

```
$ dot -Tx11 profile.dot
```

However, `xdot` is specially designed for this task:

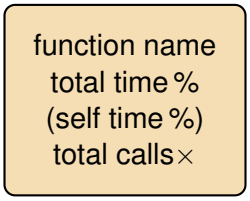
```
$ xdot profile.dot &
```

Multiple `-T` and `-o` options can be specified:

```
$ dot -Tpng -o profile.png -Tsvg -o profile.svg -Tx11 profile.dot
```

6.3 Interpreting the output

The output of `gprof2dot` consists of a labeled directed graph. Nodes represents function calls. They follow this structure:



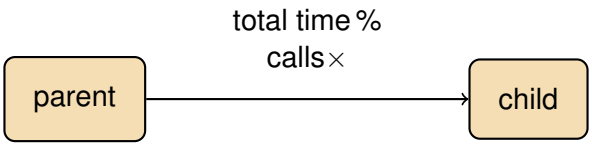
In addition to the function name, there are three relevant fields in a node:

total time %
Percentage of the running time spent in this function *and all its children*.

self time %
Percentage of the running time spent in this function *alone*.

total calls
Total number of times this function was called, *including recursive calls*.

Directed edges represent calls from a parent function (the *caller*) to a child function (the *callee*). Their layout is as follows:



We can observe two relevant fields in the edge label:

total time %
Percentage of the total time of the parent corresponding to that child.

calls
Number of times the child function was called by that parent.

Warning! By default `gprof2dot` generates a *partial* call graph. This means that it excludes nodes and edges with little or no impact in the total execution time. If we want the full call graph then the following command-line arguments can be used:

`-n`*percentage* | **`-node-thres`**=*percentage*
Node threshold. Show only nodes with a percentage of time above the threshold.

`-e`*percentage* | **`-edge-thres`**=*percentage*
Edge threshold. Show only edges with a percentage of time above the threshold.

For example, the following will set a zero threshold both for nodes and edges:

```
$ gprof main | gprof2dot -n0 -e0 | dot -Tpdf -o profile.pdf
```

Even so, if there are some nodes or edges that are not shown, maybe because the relevant information does not appear in the profile, it can be useful to accumulate more information from several runs.

6.4 Example

Following our example in Section 5, once we have obtained the profile data in `gmon.out` we can generate a graphic profile in a PDF file with the above instructions:

```
$ gprof -b main | gprof2dot -n0 -e0 | dot -Tpdf -o profile.pdf
```

As we can see in Figure 1, the information provided is consistent with that available in the flat profile and the call graph, but easier to interpret.

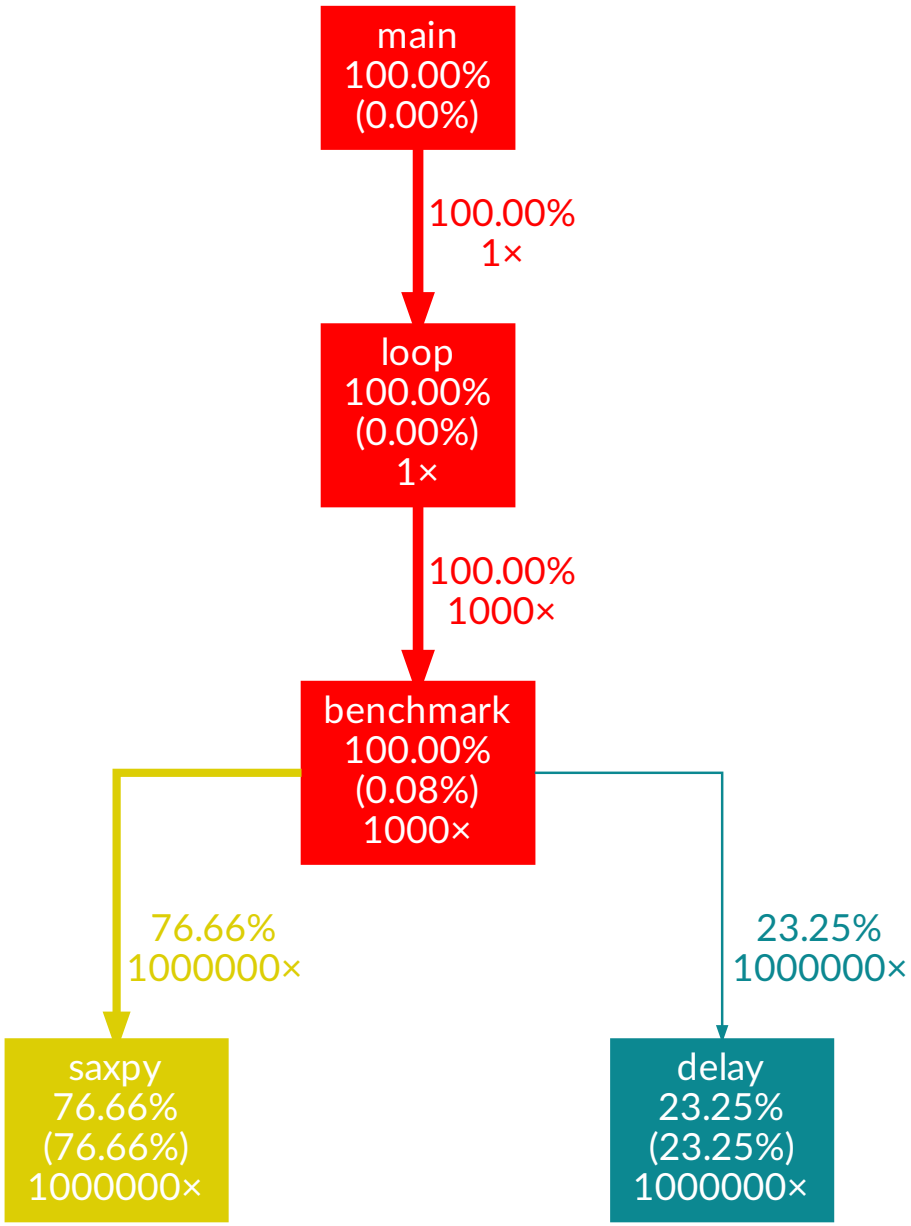


Figure 1: Graphical rendition of the profile data.